НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Інститут прикладного системного аналізу
Кафедра системного проектування

Звіт

про виконання комп'ютерного практикуму №6-7
з дисципліни «Основи молекулярної біології та біоінформатики»

студент 3го курсу
групи ДА-11
Сидько Нікіта Сергійович
Варіант 24

Київ-2024

Вирівнювання амінокислотних послідовностей. Алгоритм глобального вирівнювання Нідлмана – Вунша. Алгоритм локального вирівнювання Сміта - Уотермана.

Варіант 24

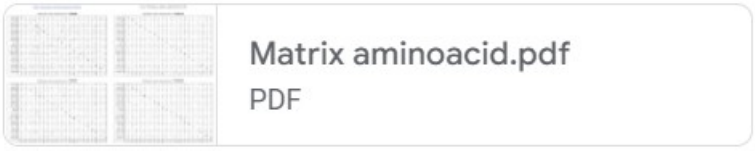
	seq1	seq2
24	IQGNLVGTGLKIGIVVGRFNDF	NLVGLKFGIVTARFNGAHDFI

24 IQGNLVGTGLKIGIVVGRFNDF NLVGLKFGIVTARFNGAHDFI

Завдання до КП

1.За допомогою матриці замін амінокислот BLOSUM 62 та фіксований штраф за пропуски d=8, методом Нідлмана - Вунша отримати глобальне вирівнювання двох фрагментів білків (дивись файл Table protein fragments. doc) та розрахувати рахунок (вагу) вирівнювання. Для вирівнювання взяти перші 5 амінокислотних залишків послідовності seq 1 та seq 2 (див. файл. protein fragment.docx).

З файлу:



Маємо наступну матрицю (BLOSUM62)

Файл: Матрицы_замен_аминокислот.pdf

Матрица замен аминокислот **BLOSUM62**

	A	R	N	D	C	Q	E	G	H	I	L	K	M	F	P	S	T	W	Y	V
Ala (A)	**4**	−1	−2	−2	0	−1	−1	0	−2	−1	−1	−1	−1	−2	−1	1	0	−3	−2	0
Arg (R)	−1	**5**	0	−2	−3	1	0	−2	0	−3	−2	2	−1	−3	−2	−1	−1	−3	−2	−3
Asn (N)	−2	0	**6**	1	−3	0	0	0	1	−3	−3	0	−2	−3	−2	1	0	−4	−2	−3
Asp (D)	−2	−2	1	**6**	−3	0	2	−1	−1	−3	−4	−1	−3	−3	−1	0	−1	−4	−3	−3
Cys (C)	0	−3	−3	−3	**9**	−3	−4	−3	−3	−1	−1	−3	−1	−2	−3	−1	−1	−2	−2	−1
Gln (Q)	−1	1	0	0	−3	**5**	2	−2	0	−3	−2	1	0	−3	−1	0	−1	−2	−1	−2
Glu (E)	−1	0	0	2	−4	2	**5**	−2	0	−3	−3	1	−2	−3	−1	0	−1	−3	−2	−2
Gly (G)	0	−2	0	−1	−3	−2	−2	**6**	−2	−4	−4	−2	−3	−3	−2	0	−2	−2	−3	−3
His (H)	−2	0	1	−1	−3	0	0	−2	**8**	−3	−3	−1	−2	−1	−2	−1	−2	−2	2	−3
Ile (I)	−1	−3	−3	−3	−1	−3	−3	−4	−3	**4**	2	−3	1	0	−3	−2	−1	−3	−1	3
Leu (L)	−1	−2	−3	−4	−1	−2	−3	−4	−3	2	**4**	−2	2	0	−3	−2	−1	−2	−1	1
Lys (K)	−1	2	0	−1	−3	1	1	−2	−1	−3	−2	**5**	−1	−3	−1	0	−1	−3	−2	−2
Met (M)	−1	−1	−2	−3	−1	0	−2	−3	−2	1	2	−1	**5**	0	−2	−1	−1	−1	−1	1
Phe (F)	−2	−3	−3	−3	−2	−3	−3	−3	−1	0	0	−3	0	**6**	−4	−2	−2	1	3	−1
Pro (P)	−1	−2	−2	−1	−3	−1	−1	−2	−2	−3	−3	−1	−2	−4	**7**	−1	−1	−4	−3	−2
Ser (S)	1	−1	1	0	−1	0	0	0	−1	−2	−2	0	−1	−2	−1	**4**	1	−3	−2	−2
Thr (T)	0	−1	0	−1	−1	−1	−1	−2	−2	−1	−1	−1	−1	−2	−1	1	**5**	−2	−2	0
Trp (W)	−3	−3	−4	−4	−2	−2	−3	−2	−2	−3	−2	−3	−1	1	−4	−3	−2	**11**	2	−3
Tyr (Y)	−2	−2	−2	−3	−2	−1	−2	−3	2	−1	−1	−2	−1	3	−3	−2	−2	2	**7**	−1
Val (V)	0	−3	−3	−3	−1	−2	−2	−3	−3	3	1	−2	1	−1	−2	−2	0	−3	−1	**4**

Метод Нідлмана-Вунша (Needleman-Wunsch) є алгоритмом для глобального вирівнювання двох послідовностей, таких як білки або нуклеотидні послідовності. Цей метод базується на принципах динамічного програмування і забезпечує оптимальне вирівнювання шляхом максимізації суми оцінок замін, вставок і видалень.
Основні етапи:

Ініціалізація матриці оцінок

Створюється двовимірна матриця розміром (n+1) x (m+1), де n і m — довжини двох послідовностей, які вирівнюються. Перший рядок і перша колонка заповнюються, враховуючи штраф за пропуски (gap penalty). Це забезпечує, що кожна клітинка матриці оцінок містить максимальну оцінку для вирівнювання до цієї точки. Формули для ініціалізації: score(i, 0) = i * gap_penalty та score(0, j) = j * gap_penalty

Заповнення матриці оцінок

Кожна клітинка матриці заповнюється на основі трьох можливих дій:
Якщо символи збігаються або є заміна, оцінка додається з матриці замін (наприклад, BLOSUM62).Вводиться штраф за пропуск у одній з послідовностей. Вводиться штраф за пропуск у іншій послідовності.
Формула для заповнення матриці виглядатиме наступним чином:
score(i, j) = max(score(i-1, j-1) + s(seq1[i-1], seq2[j-1]), score(i-1, j) + gap_penalty, score(i, j-1) + gap_penalty)
де s(x, y) — оцінка заміни для символів x і y з матриці замін.

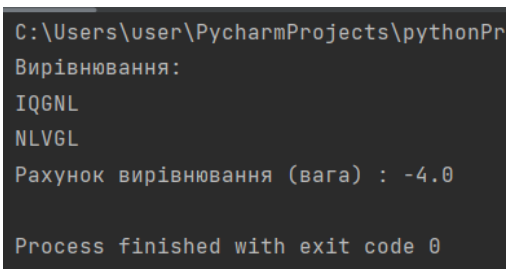
Traceback

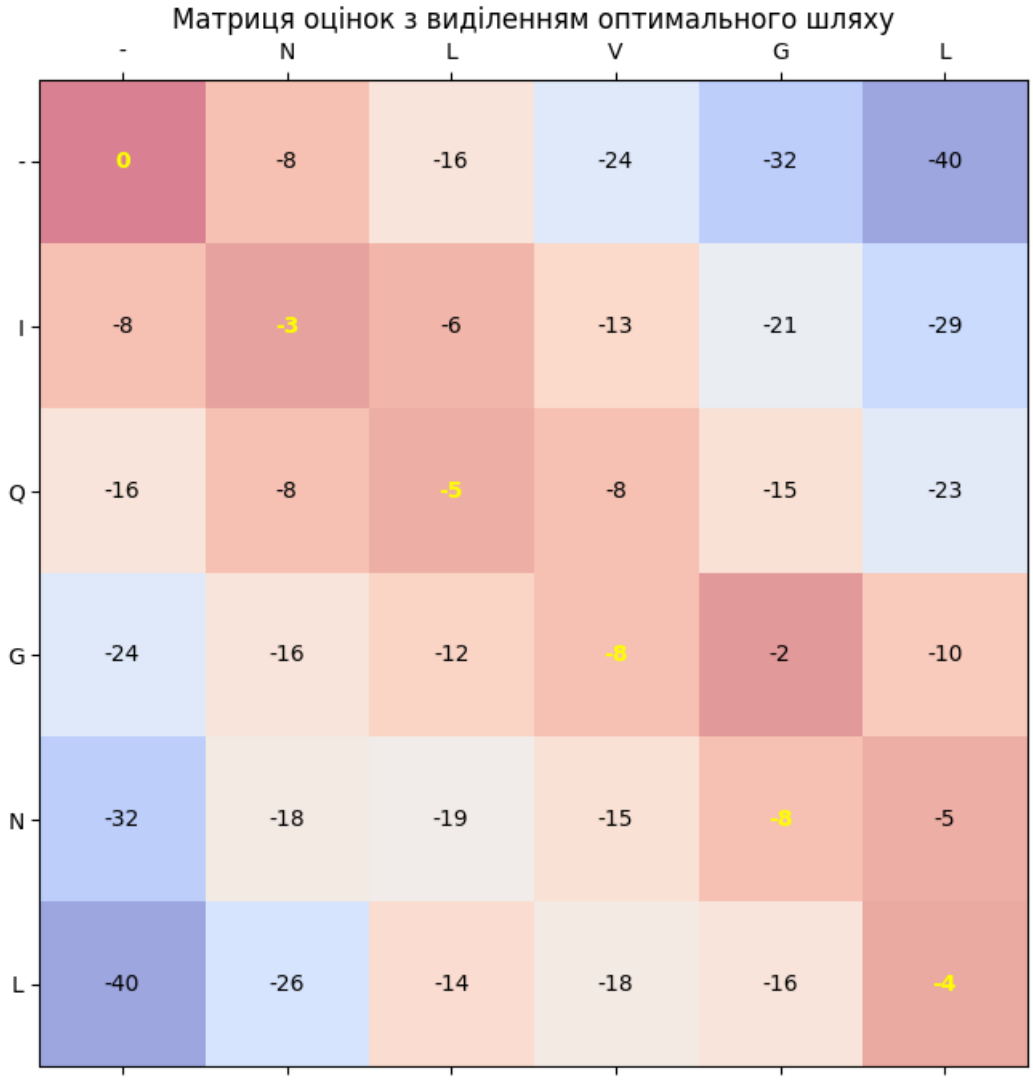
Після заповнення матриці оцінок визначається оптимальний шлях вирівнювання, починаючи з нижнього правого кута матриці і рухаючись назад до верхнього лівого кута. Цей процес слідкує за шляхом, який привів до максимальних оцінок. Формули для трейсбеку:
Якщо score(i, j) = score(i-1, j-1) + s(seq1[i-1], seq2[j-1]), то йдемо по діагоналі
Якщо score(i, j) = score(i-1, j) + gap_penalty - то йдемо вгору
Якщо score(i, j) = score(i, j-1) + gap_penalty - йдемо вліво

За варіантом маю такі дані:

seq1 – IQGNLVGTGLKIGIVVGRFNDF (**IQGNL** – перші 5 амінокисл. зал.)
seq2 – NLVGLKFGIVTARFNGAHDFI (**NLVGL** – перші 5 амінокисл. зал.)

Результат роботи коду :





Жовті цифри - клітинки, які були частиною оптимального шляху вирівнювання.

2. За допомогою матриці замін амінокислот BLOSUM 62 та фіксований штраф за пропуски d=8, методом Сміта - Уотермана побудувати локальне вирівнювання двох фрагментів білків (див. файл. protein fragment.docx) та розрахувати рахунок (вагу) вирівнювання. Для вирівнювання взяти перші 6 амінокислотних залишків послідовності seq 1 та seq 2.

Алгоритм Сміта-Уотермана - алгоритм для локального вирівнювання двох послідовностей. На відміну від алгоритму Нідлмана-Вунша, цей алгоритм знаходить найбільш схожий сегмент двох послідовностей, тобто вирівнювання з найвищою оцінкою.
Основи алгоритму Сміта-Уотермана:
1. Ініціалізація матриці оцінок

Як і в алгоритмі Нідлмана-Вунша, створюється двовимірна матриця оцінок розміром (n+1) x (m+1). Однак в алгоритмі Сміта-Уотермана всі початкові значення встановлюються рівними нулю.

2. Заповнення матриці оцінок

Кожна клітинка матриці заповнюється на основі чотирьох можливих дій: якщо символи збігаються або є заміна, оцінка додається з матриці замін (наприклад, BLOSUM62). Вводиться штраф за пропуск у одній з послідовностей.Вводиться штраф за пропуск у іншій послідовності. Якщо жодне з дій не дає позитивного результату, клітинка обнуляється (локальне вирівнювання починається з нуля). Формула для заповнення матриці:
score(i, j) = max(0, score(i-1, j-1) + s(seq1[i-1], seq2[j-1]), score(i-1, j) + gap_penalty, score(i, j-1) + gap_penalty)

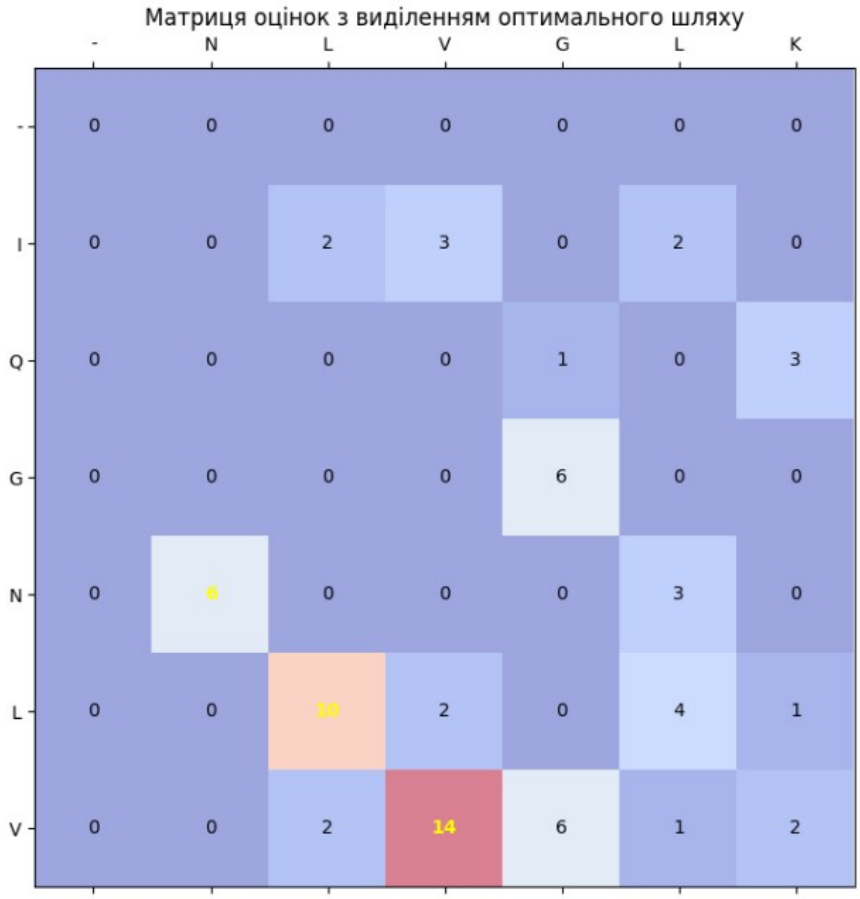
3. Traceback

Після заповнення матриці оцінок визначається оптимальний шлях вирівнювання, починаючи з клітинки з найвищою оцінкою і рухаючись назад до першої клітинки з нульовою оцінкою.

Seq1 – **IQGNLV**
Seq2 – **NLVGLK**
Результат роботи коду:

```
C:\Users\user\PycharmProjects\pythonP
Вирівнювання:
NLV
NLV
Рахунок вирівнювання (вага) : 14.0

Process finished with exit code 0
```

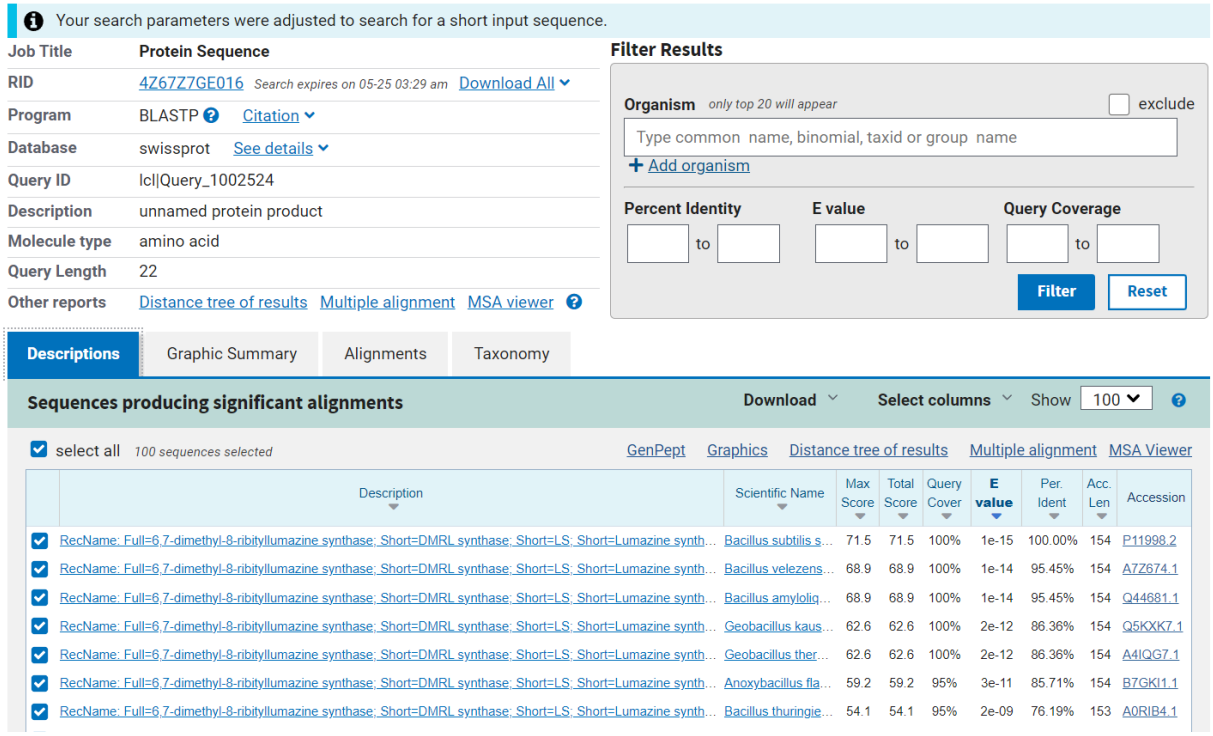


3. В яких організмах зустрічається фрагмент послідовності seq 1 та seq 2 ?

Скористаємося програмою Blast для:

Seq1 – IQGNLVGTGLKIGIVVGRFNDF

Тоді маємо:



Your search parameters were adjusted to search for a short input sequence.

Job Title	Protein Sequence
RID	4Z67Z7GE016 *Search expires on 05-25 03:29 am* Download All
Program	BLASTP Citation
Database	swissprot See details
Query ID	lcl|Query_1002524
Description	unnamed protein product
Molecule type	amino acid
Query Length	22
Other reports	Distance tree of results Multiple alignment MSA viewer

Filter Results

Organism *only top 20 will appear* exclude

Type common name, binomial, taxid or group name

+ Add organism

Percent Identity to | E value to | Query Coverage to

Filter Reset

Descriptions | Graphic Summary | Alignments | Taxonomy

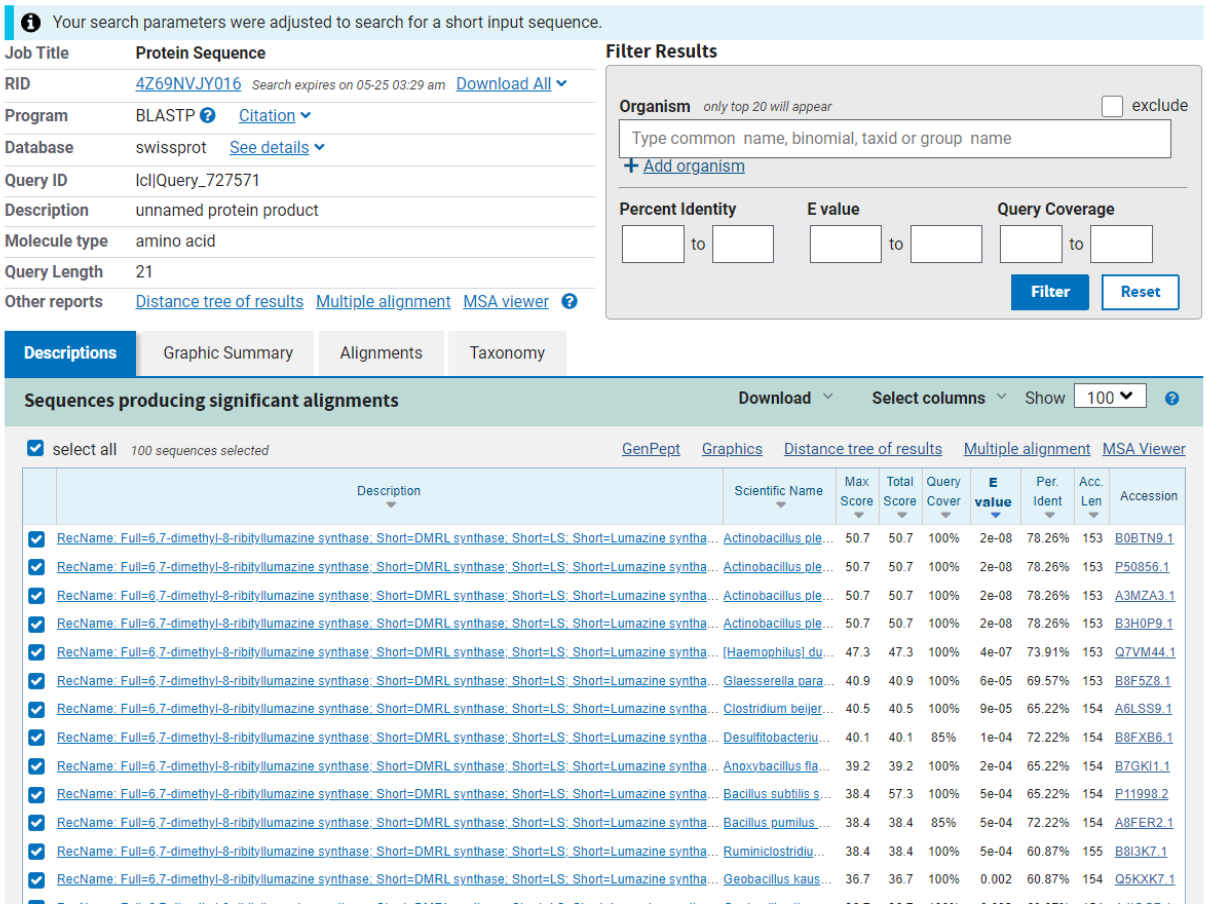
Sequences producing significant alignments — Download — Select columns — Show 100

select all *100 sequences selected* GenPept Graphics Distance tree of results Multiple alignment MSA Viewer

Description	Scientific Name	Max Score	Total Score	Query Cover	E value	Per. Ident	Acc. Len	Accession
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine synth…	Bacillus subtilis s…	71.5	71.5	100%	1e-15	100.00%	154	P11998.2
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine synth…	Bacillus velezens…	68.9	68.9	100%	1e-14	95.45%	154	A7Z674.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine synth…	Bacillus amyloliq…	68.9	68.9	100%	1e-14	95.45%	154	Q44681.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine synth…	Geobacillus kaus…	62.6	62.6	100%	2e-12	86.36%	154	Q5KXK7.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine synth…	Geobacillus ther…	62.6	62.6	100%	2e-12	86.36%	154	A4IQG7.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine synth…	Anoxybacillus fla…	59.2	59.2	95%	3e-11	85.71%	154	B7GKI1.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine synth…	Bacillus thuringie…	54.1	54.1	95%	2e-09	76.19%	153	A0RIB4.1

Тоді для :

Seq2 – NLVGLKFGIVTARFNGAHDFI



Your search parameters were adjusted to search for a short input sequence.

Job Title	Protein Sequence
RID	4Z69NVJY016 *Search expires on 05-25 03:29 am* Download All
Program	BLASTP Citation
Database	swissprot See details
Query ID	lcl|Query_727571
Description	unnamed protein product
Molecule type	amino acid
Query Length	21
Other reports	Distance tree of results Multiple alignment MSA viewer

Filter Results

Organism *only top 20 will appear* exclude

Type common name, binomial, taxid or group name

+ Add organism

Percent Identity to | E value to | Query Coverage to

Filter Reset

Descriptions | Graphic Summary | Alignments | Taxonomy

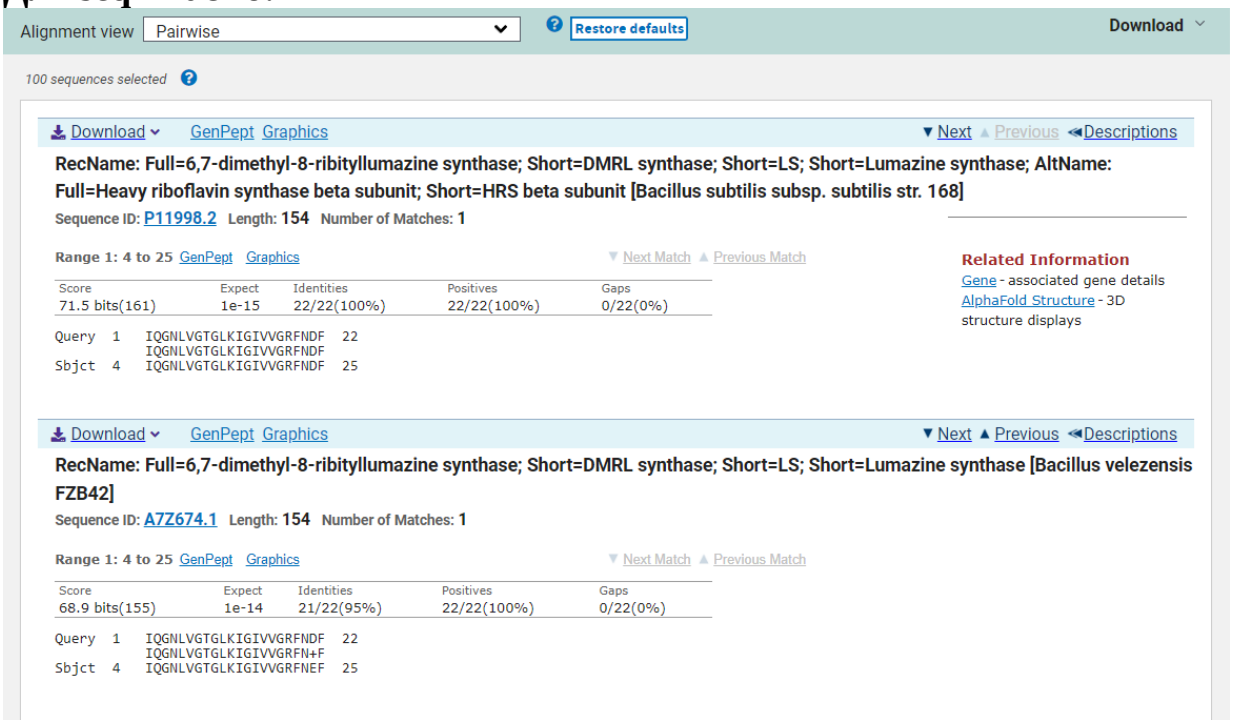
Sequences producing significant alignments — Download — Select columns — Show 100

select all *100 sequences selected* GenPept Graphics Distance tree of results Multiple alignment MSA Viewer

Description	Scientific Name	Max Score	Total Score	Query Cover	E value	Per. Ident	Acc. Len	Accession
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	Actinobacillus ple…	50.7	50.7	100%	2e-08	78.26%	153	B0BTN9.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	Actinobacillus ple…	50.7	50.7	100%	2e-08	78.26%	153	P50856.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	Actinobacillus ple…	50.7	50.7	100%	2e-08	78.26%	153	A3MZA3.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	Actinobacillus ple…	50.7	50.7	100%	2e-08	78.26%	153	B3H0P9.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	[Haemophilus] du…	47.3	47.3	100%	4e-07	73.91%	153	Q7VM44.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	Glaesserella para…	40.9	40.9	100%	6e-05	69.57%	153	B8F5Z8.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	Clostridium beijer…	40.5	40.5	100%	9e-05	65.22%	154	A6LSS9.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	Desulfitobacteriu…	40.1	40.1	85%	1e-04	72.22%	154	B8FXB6.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	Anoxybacillus fla…	39.2	39.2	100%	2e-04	65.22%	154	B7GKI1.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	Bacillus subtilis s…	38.4	57.3	100%	5e-04	65.22%	154	P11998.2
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	Bacillus pumilus…	38.4	38.4	85%	5e-04	72.22%	154	A8FER2.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	Ruminiclostridiu…	38.4	38.4	100%	5e-04	60.87%	155	B8I3K7.1
RecName: Full=6,7-dimethyl-8-ribityllumazine synthase; Short=DMRL synthase; Short=LS; Short=Lumazine syntha…	Geobacillus kaus…	36.7	36.7	100%	0.002	60.87%	154	Q5KXK7.1

4 Знайдіть інформацію про функцію білків, що містять фрагменти послідовностей seq 1 та seq 2 ?

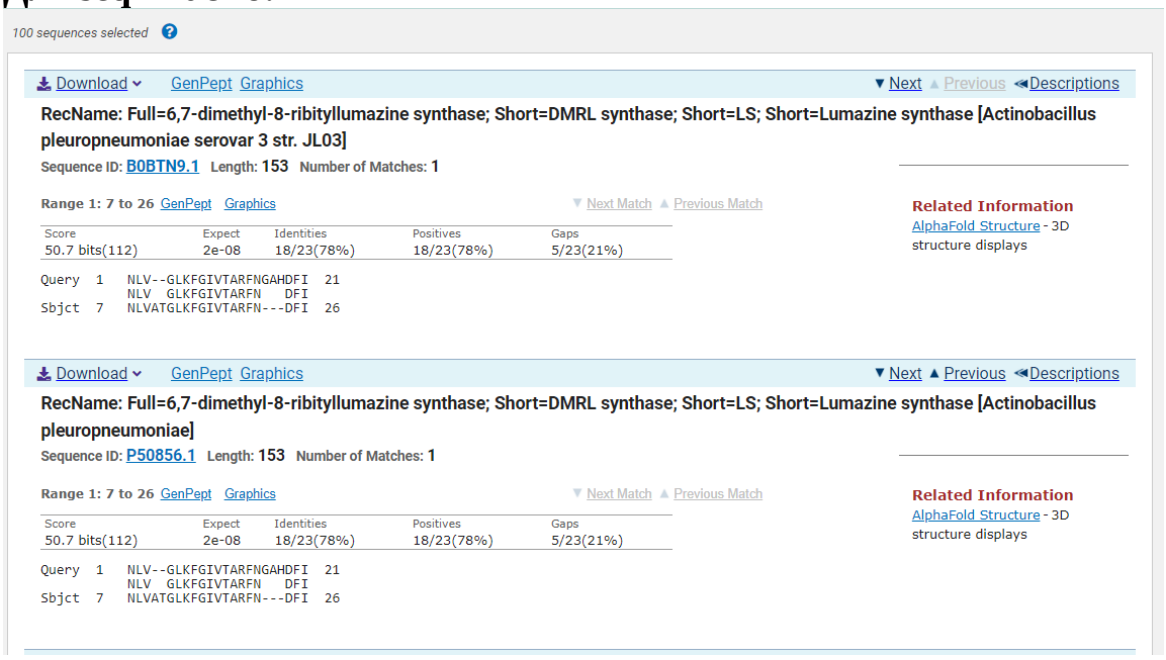
Для seq1 маємо:



Основна функція для них:

Каталізує утворення 6,7-диметил-8-рибітіллумазин конденсацією 5-аміно-6-(D-рибітиламіно)урацил з 3,4-дигідрокси-2-бутаноном 4-фосфат. Це передостанній етап біосинтезарибофлавін. Здатний використовувати неприродний R-енантіомер3,4-дигідрокси-2-бутанон-4-фосфат як субстрат, але з меншою кількістю ефективності, ніж природний енантіомер S. Не може використовувати нефосфорильований 3,4-дигідрокси-2-бутанон, 3,4-дигідрокси -2-бутанон 3-фосфат або діацетил як субстрати.

Для seq2 маємо:



Основна функція для них:
Каталізує утворення 6,7-диметил-8-рібітиллумазин шляхом конденсації 5-аміно-6-(D-рибітиламіно)урацил з 3,4-дигідрокси-2-бутаноном 4-фосфат. Це передостанній етап біосинтезу рибофлавін.

5. Знайдені білки будуть гомологами чи ні ?
Для цього скористаємося needle:
Fasta1:

```
Fasta1: >sp|P11998.2|RISB_BACSU RecName: Full=6,7-dimethyl-8-ribityllumazine
synthase; Short=DMRL synthase; Short=LS; Short=Lumazine synthase; AltName:
Full=Heavy riboflavin synthase beta subunit; Short=HRS beta subunit
MNIIQGNLVGTGLKIGIVVGRFNDFITSKLLSGAEDALLRHGVDTNDIDVAWVPGAFEIPFAAKKMAETK
KYDAIITLGTVIRGATTHYDYVCNEAAKGIAQAANTTGVPVIFGIVTTENIEQAIERAGTKAGNKGVDCA
VSAIEMANLNRSFE
```

Fasta2:

```
>sp|B0BTN9.1|RISB_ACTPJ RecName: Full=6,7-dimethyl-8-ribityllumazine
synthase; Short=DMRL synthase; Short=LS; Short=Lumazine synthase
MAKITGNLVATGLKFGIVTARFNDFINDKLLSGAIDTLVRHGADENNIDTAWVPGAFEIPLVAKKMATSG
KYDAVICLGTVIRGSTTHYDYVCNEAAKGIGAVALETGVPVIFGVLTTENIEQAIERAGTKAGNKGSECA
LGAIEIVNVLKAI
```

Результат:

```
#=======================================
#
# Aligned_sequences: 2
# 1: RISB_BACSU
# 2: RISB_ACTPJ
# Matrix: EBLOSUM62
# Gap_penalty: 10.0
# Extend_penalty: 0.5
#
# Length: 154
# Identity:     111/154 (72.1%)
# Similarity:   124/154 (80.5%)
# Gaps:           1/154 ( 0.6%)
# Score: 573.0
#
#
#=======================================
```

Висновок.

У ході виконання лабораторної роботи 6-7 були розглянуті алгоритми (метод Сміта – Уотермана) локальне вирівнювання, та (метод Нідлмана – Вунша) глобальне вирівнювання, та реалізовані за допомогою мови Python. За допомогою Blast, знайшли білки, у яких міститься seq1 та seq2, після чого було зроблено тестування чи є вони гомологами, за допомогою needle.

Лістинг коду (глобальне вирівнювання):

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from Bio.Align import substitution_matrices
```

```
blosum62 = substitution_matrices.load("BLOSUM62")

seq1 = "IQGNL"
seq2 = "NLVGL"

gap_penalty = -8
n = len(seq1) + 1
m = len(seq2) + 1
score_matrix = np.zeros((n, m))
traceback_matrix = np.zeros((n, m), dtype=str)

for i in range(1, n):
    score_matrix[i][0] = score_matrix[i - 1][0] + gap_penalty
    traceback_matrix[i][0] = '↑'
for j in range(1, m):
    score_matrix[0][j] = score_matrix[0][j - 1] + gap_penalty
    traceback_matrix[0][j] = '←'

for i in range(1, n):
    for j in range(1, m):
        match = score_matrix[i - 1][j - 1] + blosum62[seq1[i - 1], seq2[j - 1]]
        delete = score_matrix[i - 1][j] + gap_penalty
        insert = score_matrix[i][j - 1] + gap_penalty
        score_matrix[i][j] = max(match, delete, insert)

        if score_matrix[i][j] == match:
            traceback_matrix[i][j] = '↖'
        elif score_matrix[i][j] == delete:
            traceback_matrix[i][j] = '↑'
        else:
            traceback_matrix[i][j] = '←'

align1 = ""
align2 = ""
i = n - 1
j = m - 1

highlight = np.zeros_like(score_matrix, dtype=bool)

while i > 0 or j > 0:
    highlight[i, j] = True
    if traceback_matrix[i][j] == '↖':
        align1 = seq1[i - 1] + align1
        align2 = seq2[j - 1] + align2
        i -= 1
        j -= 1
    elif traceback_matrix[i][j] == '↑':
        align1 = seq1[i - 1] + align1
        align2 = '-' + align2
        i -= 1
    else:
        align1 = '-' + align1
        align2 = seq2[j - 1] + align2
        j -= 1
highlight[0, 0] = True

alignment_score = score_matrix[n - 1][m - 1]

print("Вирівнювання:")
print(align1)
print(align2)
print("Рахунок вирівнювання (вага) :", alignment_score)

fig, ax = plt.subplots(figsize=(10, 8))
ax.matshow(score_matrix, cmap='coolwarm', alpha=0.5)
```

```
for i in range(score_matrix.shape[0]):
    for j in range(score_matrix.shape[1]):
        if highlight[i, j]:
            ax.text(j, i, int(score_matrix[i, j]), va='center', ha='center',
color='yellow', fontweight='bold')
        else:
            ax.text(j, i, int(score_matrix[i, j]), va='center', ha='center',
color='black')

plt.xticks(np.arange(m), ["-"] + list(seq2))
plt.yticks(np.arange(n), ["-"] + list(seq1))
plt.title("Матриця оцінок з виділенням оптимального шляху")
plt.show()
```

Лістинг коду (локальне вирівнювання):

```
import numpy as np
import matplotlib.pyplot as plt
from Bio.Align import substitution_matrices

blosum62 = substitution_matrices.load("BLOSUM62")

seq1 = "IQGNLV"
seq2 = "NLVGLK"

gap_penalty = -8
n = len(seq1) + 1
m = len(seq2) + 1
score_matrix = np.zeros((n, m))
traceback_matrix = np.zeros((n, m), dtype=str)

max_score = 0
max_pos = None

for i in range(1, n):
    for j in range(1, m):
        match = score_matrix[i - 1][j - 1] + blosum62[seq1[i - 1], seq2[j - 1]]
        delete = score_matrix[i - 1][j] + gap_penalty
        insert = score_matrix[i][j - 1] + gap_penalty
        score_matrix[i][j] = max(0, match, delete, insert)

        if score_matrix[i][j] == match:
            traceback_matrix[i][j] = '↖'
        elif score_matrix[i][j] == delete:
            traceback_matrix[i][j] = '↑'
        elif score_matrix[i][j] == insert:
            traceback_matrix[i][j] = '←'
        else:
            traceback_matrix[i][j] = '0'

        if score_matrix[i][j] >= max_score:
            max_score = score_matrix[i][j]
            max_pos = (i, j)

align1 = ""
align2 = ""
i, j = max_pos

highlight = np.zeros_like(score_matrix, dtype=bool)

while score_matrix[i][j] != 0:
    highlight[i, j] = True
    if traceback_matrix[i][j] == '↖':
```

```
            align1 = seq1[i - 1] + align1
            align2 = seq2[j - 1] + align2
            i -= 1
            j -= 1
        elif traceback_matrix[i][j] == '↑':
            align1 = seq1[i - 1] + align1
            align2 = '-' + align2
            i -= 1
        elif traceback_matrix[i][j] == '←':
            align1 = '-' + align1
            align2 = seq2[j - 1] + align2
            j -= 1

    highlight[max_pos] = True

    alignment_score = max_score

    print("Вирівнювання:")
    print(align1)
    print(align2)
    print("Рахунок вирівнювання (вага) :", alignment_score)

    fig, ax = plt.subplots(figsize=(10, 8))
    ax.matshow(score_matrix, cmap='coolwarm', alpha=0.5)

    for i in range(score_matrix.shape[0]):
        for j in range(score_matrix.shape[1]):
            if highlight[i, j]:
                ax.text(j, i, int(score_matrix[i, j]), va='center', ha='center',
    color='yellow', fontweight='bold')
            else:
                ax.text(j, i, int(score_matrix[i, j]), va='center', ha='center',
    color='black')

    plt.xticks(np.arange(m), ["-"] + list(seq2))
    plt.yticks(np.arange(n), ["-"] + list(seq1))
    plt.title("Матриця оцінок з виділенням оптимального шляху")
    plt.show()
```